UNIT 4 · TOPIC 4.4

Detecting Attacks on Devices

Assignment, Answer Key

Teacher answer key, 33 points total.

Practice Questions

4.4.a · Endpoint detection and response (EDR)

1. What distinguishes EDR from traditional antivirus software? (1 pt)

- ○ A. EDR only checks files against known signatures at rest
- **● B. EDR monitors continuous runtime behavior and can respond by isolating or killing processes**
- ○ C. EDR runs only at the network perimeter, not on individual devices
- ○ D. EDR requires internet access to query a cloud threat database

Why: EDR goes beyond signature scanning: it watches live process, file, network, and registry events and can respond automatically (isolate, kill, quarantine), unlike antivirus which primarily scans static files.

2. Which of the following are telemetry streams that an EDR agent typically collects? Select ALL that apply. (3 pts)

- **■ A. Process creation and termination events**
- **■ B. File system changes (writes, renames, deletes)**
- □ C. Physical room temperature from HVAC sensors
- **■ D. Network connections opened by each process**

Why: EDR collects process events, file-system events, registry events, and network connection events from each endpoint. HVAC sensors are not endpoint telemetry.

3. An attacker uses PowerShell (a legitimate Windows tool) to download and run malware. Which EDR capability is MOST likely to catch this, and why? (1 pt)

- ○ A. Signature matching, because PowerShell is on the known-bad list
- ○ B. Endpoint isolation, because PowerShell always triggers isolation
- ○ C. File quarantine, because the PowerShell binary itself is quarantined
- **● D. Behavioral detection, because the unusual parent-child process relationship and outbound connection deviate from the baseline**

Why: Living-off-the-land attacks use legitimate tools, so no signature fires. Behavioral detection flags the deviation from normal patterns (e.g., Office spawning PowerShell, PowerShell opening an outbound connection).

4. When an EDR platform 'isolates' an endpoint, what does that mean? (1 pt)

- ○ A. The device's hard drive is wiped immediately
- **● B. All network connections are cut except the EDR management channel, stopping lateral movement**
- ○ C. The user is logged out and their account is disabled permanently
- ○ D. The device is powered off remotely

Why: Isolation cuts all network access except the EDR console channel, preventing the attacker from moving to other systems while the analyst can still investigate remotely.

5. Name the attacker technique where legitimate operating-system tools like PowerShell or WMI are misused to carry out malicious actions. (1 pt)

Acceptable answers: living off the land; living-off-the-land; lolbins; lol bins; lol-bins

Why: Living off the land (also abbreviated LoLBins) describes using built-in OS tools for malicious purposes, making detection harder because no foreign malware binary appears on disk.

6. Why is it incorrect to assume a device is clean just because no EDR alert has fired? (1 pt)

○ A. EDR only scans files when they are first downloaded

○ B. EDR alerts are hidden from analysts until they are confirmed

● C. EDR detection relies on signatures and baselines that may not cover every possible attack

○ D. EDR stops monitoring a device once it has been scanned clean

Why: No detection tool sees everything: new techniques, careful attackers who blend in with normal activity, or gaps in what the agent records can all produce false negatives. "No alert" means "nothing matched our detections", not "nothing happened", which is why teams also do threat hunting.

4.4.b · Log analysis

7. Why is a single failed login log entry usually NOT enough evidence of an attack? (1 pt)

○ A. Failed logins are only logged when the account is locked

○ B. Attackers never fail more than once because they use stolen passwords

○ C. A single failure is ordinary human error; attacks are recognized through correlated patterns across multiple entries

● D. Single entries are compressed and cannot be read until more arrive

Why: Everyone mistypes passwords. One failure means almost nothing; what matters is the pattern: many failures, many accounts, unusual sources, or a failure streak followed by a success. That is why analysts correlate entries instead of reacting to each one.

8. What does 'correlation' mean in the context of log analysis? (1 pt)

○ A. Comparing a log entry to a list of known-bad IP addresses

● B. Linking multiple log entries together (by user, IP, or time) to reveal a pattern invisible in any single entry

○ C. Calculating the average number of log entries per hour

○ D. Sorting log entries by severity so the worst appear first

Why: Correlation connects events from different places or times: the same IP failing logins on three servers, or a VPN login followed by a new admin account. Each event alone looks routine; together they reveal an attack. Checking an IP against a blocklist is matching, not correlation.

9. Select ALL statements that are TRUE about SIEM platforms. (3 pts)

■ A. They ingest logs from many systems across an organization

■ B. They apply correlation rules automatically at scale to raise alerts

□ C. They eliminate the need for any human security analyst

■ D. They can generate false positives if detection thresholds are set too sensitively

Why: SIEMs centralize and correlate logs at scale, but still require analysts to investigate alerts; overly sensitive rules still cause false positives.

10. An authentication log shows 6 failed logins for the same user within 45 seconds, followed immediately by a success from the same IP. What does this pattern most likely indicate? (1 pt)

○ A. The user simply forgot their password and does not need investigation

● B. A brute-force attack that guessed the password after repeated attempts

○ C. A hardware failure on the authentication server

○ D. Normal end-of-day login behavior

Why: Multiple rapid failed attempts followed by a success from the same source is the textbook signature of a brute-force attack succeeding.

11. Name the phenomenon where overly sensitive detection thresholds cause so many false alarms that analysts start ignoring alerts, potentially missing real attacks. (1 pt)

Acceptable answers: alert fatigue; alert-fatigue

Why: Alert fatigue occurs when detection rules are tuned too sensitively, generating excessive false positives that desensitize analysts.

12. Why are logs valuable even after an attacker has already left a compromised system? (1 pt)

○ A. Logs let the team reverse the attacker's changes automatically

● **B. Logs are the record used for forensic reconstruction of what the attacker did**

○ C. Logs prove the attacker's identity in court on their own

○ D. Logs show which antivirus product failed

Why: After an intrusion, logs are the timeline: when the attacker got in, which accounts they used, what they touched and where data went. That drives cleanup, notification and lessons learned. Logs support evidence but rarely prove identity alone, and they do not undo changes.

4.4.c · Identifying compromised devices (indicators of compromise)

13. What is an indicator of compromise (IOC)? (1 pt)

○ A. Any alert raised by antivirus software

○ B. A known vulnerability on a device that has not been patched

● **C. Forensic evidence (a file, connection, account, or behavior) suggesting a device has been breached**

○ D. A list of the attacker's names and locations

Why: An IOC is a trace left by an intrusion: a malicious file hash, a connection to a known C2 server, a mystery admin account, a disabled security tool. A missing patch is a vulnerability (a weakness that MIGHT be used), not evidence that it was used.

14. Why is a single IOC usually not enough to confidently declare a device compromised? (1 pt)

○ A. IOCs expire after 24 hours and must be re-collected

○ B. Attackers always leave at least three IOCs, so one means a false alarm

○ C. IOCs only count once they appear in a threat-intelligence feed

● **D. A single artifact often has an innocent explanation; confident verdicts come from combining multiple independent IOCs**

Why: A new admin account might be IT onboarding; a disabled antivirus might be an update. Each artifact alone has innocent explanations, but several independent ones pointing the same way make compromise likely, which is why the weighted scoring in this skill adds them up.

15. Select ALL examples of host-based indicators of compromise. (Choose all that apply.) (3 pts)

■ **A. A new administrator account nobody remembers creating**

■ **B. A registry run-key added for persistence**

■ **C. Antivirus/EDR protection silently disabled**

□ D. A DNS lookup for an algorithmically generated domain name

Why: New admin accounts, registry persistence, and disabled security tooling are host-based IOCs. Algorithmically generated domain lookups are a network-based IOC.

16. Using weight table: disabled AV = 5, new admin account = 4, thresholds clean<4, suspicious 4-7, compromised>=8. A device shows BOTH IOCs. What is the verdict? (1 pt)

- ○ A. Clean, because neither IOC alone reaches 8
- ○ B. Suspicious, because two IOCs together should never be scored as fully compromised
- ○ C. Cannot be determined without more information
- **● D. Compromised, because the combined score (9) is at or above the compromised threshold**

Why: 5 + 4 = 9, which is >= 8, the compromised threshold. Weighted scores combine across multiple IOCs, not just the strongest single one.

17. Name the attacker technique of maintaining access to a compromised system even after the initial foothold is discovered or the system is rebooted. (1 pt)

Acceptable answers: persistence

Why: Persistence describes techniques (like a new admin account or a registry run key) that let an attacker regain access even after remediation attempts.

18. How does an indicator of attack (IOA) differ from an indicator of compromise (IOC)? (1 pt)

- ○ A. IOAs come from antivirus, while IOCs come from firewalls
- ○ B. IOAs are used only by governments, while IOCs are used by companies
- **● C. An IOA focuses on catching attacker technique/intent while an attack is in progress; an IOC is typically after-the-fact evidence**
- ○ D. IOAs describe vulnerabilities, while IOCs describe patches

Why: An IOC is a fingerprint left behind (a hash, an address, an account) and usually confirms a breach after the fact. An IOA describes behavior in progress, like Word launching PowerShell that then contacts the internet, so defenders can stop an attack even when the specific files are new.

Coding Assignments (Python 3)

4.4.a · Endpoint detection and response (EDR)

3 pts · auto-graded on the site

Coding problem 4.4.a: EDR telemetry analyzer

Skill: Detect Attacks · **Maps to:** 4.4.A · **Language:** Python 3 (pytest)

Problem

An EDR agent streams process-event telemetry from an endpoint. Each event records a process, its parent process, whether it opened an external network connection, and how many bytes it transferred. Build the analyzer that flags processes exhibiting either of the two classic red flags an EDR platform's behavioral detection looks for: a "living off the land" parent-child relationship, or unusual outbound data transfer.

Function contract

Implement `def solve(events: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dicts, each with keys `pid` (int), `process` (str), `parent_process` (str), `external_connection` (bool), `bytes_transferred` (int).

Output

A sorted list of `pid` (int) for every event that is flagged.

Detection rules (a process is flagged if EITHER rule fires)

1. **Living-off-the-land parent-child pair:** `process` (case-insensitive) is one of the script interpreters {`"powershell.exe"`, `"cmd.exe"`, `"wscript.exe"`, `"cscript.exe"`, `"mshta.exe"`} AND `parent_process` (case-insensitive) is **not** `"explorer.exe"` (the only parent considered a normal, user-initiated launch of an interpreter). Any other parent, including `winword.exe`, `outlook.exe`, `svchost.exe`, or a browser, is treated as suspicious, matching the living-off-the-land pattern where a document or script silently spawns a shell. 2. **Unusual outbound transfer:** `external_connection` is `True` AND `bytes_transferred > 1000000` (strictly greater than 1,000,000 bytes).

Examples

```
[{"pid": 501, "process": "powershell.exe", "parent_process": "winword.exe",
  "external_connection": True, "bytes_transferred": 500},
 {"pid": 502, "process": "chrome.exe", "parent_process": "explorer.exe",
  "external_connection": True, "bytes_transferred": 200000},
 {"pid": 503, "process": "backup.exe", "parent_process": "explorer.exe",
  "external_connection": True, "bytes_transferred": 5000000}]
    -> [501, 503]
```

(pid 501 flagged for the living-off-the-land parent; pid 503 flagged for excessive outbound transfer; pid 502 is a normal browser launch under the transfer threshold.)

Constraints

- Deterministic, offline, standard library only.
- Case-insensitive matching of process/parent names.
- Return a de-duplicated, ascending-sorted list of pids (each pid appears at most once even if both rules fire).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
INTERPRETERS = {"powershell.exe", "cmd.exe", "wscript.exe", "cscript.exe", "mshta.exe"}
SAFE_PARENT = "explorer.exe"
TRANSFER_THRESHOLD = 1000000


def solve(events: list) -> list:
    flagged = set()
    for ev in events:
        pid = ev["pid"]
        process = ev.get("process", "").lower()
        parent = ev.get("parent_process", "").lower()
        external = ev.get("external_connection", False)
        transferred = ev.get("bytes_transferred", 0)

        if process in INTERPRETERS and parent != SAFE_PARENT:
            flagged.add(pid)
            continue
        if external and transferred > TRANSFER_THRESHOLD:
            flagged.add(pid)
```

```
            continue

    return sorted(flagged)
```

4.4.b · Log analysis

3 pts · auto-graded on the site

Coding problem 4.4.b: Brute-force log correlator

Skill: Detect Attacks · **Maps to:** 4.4.B · **Language:** Python 3 (pytest)

Problem

An authentication log records every login attempt as an event with a username, a timestamp (in seconds), and a result (`"SUCCESS"` or `"FAILED"`). A brute-force attack shows up as a cluster of failed attempts for the same user, tightly packed in time. Build the correlator that scans the log and flags every username showing this pattern.

Function contract

Implement `def solve(entries: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of dicts, each with keys `ts` (int, seconds since some reference point), `user` (str), `result` (str, `"SUCCESS"` or `"FAILED"`). Entries are not guaranteed to already be sorted by timestamp.

Output

A sorted list of usernames (str) flagged for a brute-force pattern.

Detection rule

A user is flagged if there exist **5 or more `"FAILED"` entries for that user** where the time span from the earliest to the latest of those 5 (or more) entries is **at most 60 seconds** (inclusive). In other words: sort that user's failed-attempt timestamps ascending; if any 5 consecutive failed timestamps (in that sorted order) span 60 seconds or less (`timestamps[i+4] - timestamps[i] <= 60`), the user is flagged. SUCCESS entries do not count toward the failure count and do not reset or break a streak of failures, only FAILED entries matter for this rule.

Examples

```
[{"ts": 0, "user": "jsmith", "result": "FAILED"},
 {"ts": 13, "user": "jsmith", "result": "FAILED"},
 {"ts": 21, "user": "jsmith", "result": "FAILED"},
 {"ts": 30, "user": "jsmith", "result": "FAILED"},
 {"ts": 38, "user": "jsmith", "result": "FAILED"},
 {"ts": 46, "user": "jsmith", "result": "SUCCESS"},
 {"ts": 100, "user": "agarcia", "result": "FAILED"},
 {"ts": 1662, "user": "agarcia", "result": "FAILED"}]
    -> ["jsmith"]
```

(jsmith has 5 failures spanning 0-38s = 38s <= 60s -> flagged. agarcia has only 2 failures, 1562s apart -> not flagged.)

Constraints

- Deterministic, offline, standard library only.
- Input entries may arrive out of timestamp order; sort within your implementation.
- Return a de-duplicated, alphabetically sorted list of usernames.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
WINDOW_SECONDS = 60
MIN_FAILURES = 5


def solve(entries: list) -> list:
    failures_by_user = {}
    for entry in entries:
        if entry.get("result") == "FAILED":
            user = entry["user"]
            failures_by_user.setdefault(user, []).append(entry["ts"])

    flagged = set()
    for user, timestamps in failures_by_user.items():
        timestamps.sort()
        if len(timestamps) < MIN_FAILURES:
            continue
        for i in range(len(timestamps) - MIN_FAILURES + 1):
            span = timestamps[i + MIN_FAILURES - 1] - timestamps[i]
            if span <= WINDOW_SECONDS:
                flagged.add(user)
                break

    return sorted(flagged)
```

4.4.c · Identifying compromised devices (indicators of compromise)

3 pts · auto-graded on the site

Coding problem 4.4.c: Weighted IOC-scoring engine

Skill: Detect Attacks · **Maps to:** 4.4.B, 4.4.C · **Language:** Python 3 (pytest)

Problem

Each device under investigation reports a set of observed indicators of compromise (IOCs). Each IOC type carries a fixed severity weight. Build the scoring engine that sums the weights of the IOCs observed on a device and returns an overall compromise verdict.

Function contract

Implement `def solve(iocs: list) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

A list of strings, each an IOC type observed on one device. Duplicate entries in the list count only once (an IOC type either is or is not present on the device).

IOC weight table

IOC type string	weight
`"known_malicious_hash"`	6
`"av_disabled"`	5
`"new_admin_account"`	4
`"unusual_outbound_traffic"`	3
`"unexpected_scheduled_task"`	3
`"registry_run_key_added"`	2
`"failed_login_cluster"`	1

Any IOC type string not in this table contributes a weight of 0 (it is ignored, not an error).

Output

One of the strings: `"clean"` (total score < 4), `"suspicious"` (4 <= total score < 8), `"compromised"` (total score >= 8).

Examples

```
["av_disabled", "new_admin_account"]
    -> "compromised"   # 5 + 4 = 9 >= 8

["failed_login_cluster"]
    -> "clean"          # 1 < 4

["new_admin_account", "failed_login_cluster"]
    -> "suspicious"      # 4 + 1 = 5, in [4, 8)
```

Constraints

- Deterministic, offline, standard library only.
- Duplicate IOC types in the input list must not be double-counted.
- Unknown/unrecognized IOC type strings contribute 0 and must not raise an error.
- Thresholds are: `< 4` clean, `4 <= score < 8` suspicious, `>= 8` compromised.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
WEIGHTS = {
    "known_malicious_hash": 6,
    "av_disabled": 5,
    "new_admin_account": 4,
    "unusual_outbound_traffic": 3,
    "unexpected_scheduled_task": 3,
    "registry_run_key_added": 2,
    "failed_login_cluster": 1,
}


def solve(iocs: list) -> str:
    unique_iocs = set(iocs)
    score = sum(WEIGHTS.get(ioc, 0) for ioc in unique_iocs)

    if score >= 8:
        return "compromised"
    if score >= 4:
        return "suspicious"
    return "clean"
```